🚀 Git Fundamentals - Mastering Version Control

![Git Logo](https://git-scm.com/images/logos/downloads/Git-Icon-1788C.png)

🔹 **Nivel**: Fundamental a Avanzado
🔹 **Tiempo estimado de estudio**: 2 - 4 horas
🔹 **Aplicación**: Entrevistas Técnicas / Desarrollo de Software / Colaboración en Equipos

Bienvenido a la **guía definitiva de Git**, diseñada para entrevistas técnicas y como referencia práctica para ingenieros de software. Aquí encontrarás preguntas frecuentes de entrevistas junto con respuestas detalladas y ejemplos de código.

> _"El código sin control de versiones es como construir sin un plano."_ - Todos los ingenieros de Silicon Valley

---

📌 Índice

1. ¿Qué es un Sistema de Control de Versiones?
2. ¿Qué es Git?
3. Diferencias entre Git y GitHub
4. Comandos esenciales de Git
5. ¿Qué es un Merge Conflict y cómo solucionarlo?
6. ¿Qué es Git Stash y cómo usarlo?
7. Git Merge vs Git Rebase
8. Recursos adicionales

🔹 ¿Qué es un Sistema de Control de Versiones?

Un **Sistema de Control de Versiones (VCS)** es una herramienta que permite realizar un seguimiento de los cambios en el código fuente a lo largo del tiempo. Existen dos tipos principales:

1. **Version Control Centralizado (CVCS)** → Un solo servidor central almacena todas las versiones (Ejemplo: SVN, Perforce).
2. **Version Control Distribuido (DVCS)** → Cada usuario tiene una copia completa del repositorio (Ejemplo: Git, Mercurial).

🔥 **Ejemplo Visual:**

```text
Desarrollo sin VCS:
```

```
   - archivo_v1.js
   - archivo_v2_final.js
   - archivo_v3_ahora_sí_final.js

Desarrollo con Git:
   - archivo.js (seguimiento con commits y branches)
```

🔹 ¿Qué es Git?

Git es un **Sistema de Control de Versiones Distribuido (DVCS)** diseñado para manejar proyectos de software con eficiencia y velocidad. Es usado en todo el mundo por empresas como **Google, Microsoft, Netflix y Facebook**.

🔥 Características clave

✔ **Distribuido** → Cada usuario tiene una copia completa del historial del proyecto.
✔ **Velocidad** → Git es extremadamente rápido en comparación con otros VCS.
✔ **Seguridad** → Cada cambio es registrado de forma segura mediante SHA-1.
✔ **Flujo de trabajo flexible** → Git permite diversas estrategias de colaboración como **Git Flow** o **GitHub Flow**.

🔹 **Comando para verificar la instalación de Git:**

```sh
git --version
```

🔹 Diferencias entre Git y GitHub

Característica	Git	GitHub
¿Qué es?	Un sistema de control de versiones	Plataforma de alojamiento de repositorios Git
Almacenamiento	Local y distribuido	En la nube
Uso principal	Control de versiones de código	Colaboración y gestión de proyectos
Accesibilidad	Línea de comandos y GUI	Interfaz web y Git CLI

💡 **GitHub no es Git** → GitHub es solo una plataforma que **alberga** proyectos que usan Git.

🔹 Comandos esenciales de Git

Aquí tienes una lista de comandos fundamentales:

Comando	Descripción
`git init`	Inicializa un nuevo repositorio Git
`git clone <repo>`	Clona un repositorio existente
`git status`	Muestra el estado del repositorio
`git add <archivo>`	Agrega archivos al área de staging
`git commit -m "Mensaje"`	Confirma los cambios en el historial
`git push origin main`	Sube los cambios al repositorio remoto
`git pull origin main`	Descarga los cambios más recientes

🔥 **Ejemplo de uso en un flujo de trabajo real:**

```sh
git init
git add .
git commit -m "Primer commit"
git branch -M main
git remote add origin https://github.com/user/repo.git
git push -u origin main
```

🔹 ¿Qué es un Merge Conflict y cómo solucionarlo?

Un **Merge Conflict** ocurre cuando Git no puede fusionar automáticamente cambios en el mismo archivo desde diferentes ramas.

🔥 **Ejemplo de Conflicto:**

```sh
<<<<<<< HEAD
console.log("Hola mundo desde la rama Main");
=======
console.log("Hola mundo desde la rama Feature");
>>>>>>> feature-branch
```

💡 **Solución:**

1. Editar manualmente el archivo para elegir qué cambios conservar.
2. Agregar el archivo corregido con `git add archivo.js`.
3. Confirmar los cambios con `git commit -m "Resolviendo conflicto"`.

🔹 ¿Qué es Git Stash y cómo usarlo?

El comando `git stash` permite **guardar cambios temporalmente** sin comprometerlos en un commit.

🔥 **Ejemplo de uso:**

```sh
git stash          # Guarda los cambios
git stash list     # Muestra los stashes guardados
git stash pop      # Restaura el último stash
```

💡 **Situación común:**
Tienes cambios locales, pero necesitas cambiar de rama sin perder tu trabajo. `git stash` guarda tu progreso y puedes recuperarlo después.

🔹 Git Merge vs Git Rebase

Acción	Git Merge	Git Rebase
¿Qué hace?	Combina ramas manteniendo el historial	Aplica cambios sobre una rama base
Historial	Puede volverse complejo con múltiples merges	Se mantiene más limpio
Uso recomendado	Trabajo colaborativo en equipos grandes	Mantener un historial lineal y ordenado

🔥 **Ejemplo de Git Rebase:**

```sh
git checkout feature-branch
git rebase main
```

Esto mueve la `feature-branch` sobre `main`, evitando commits de merge innecesarios.

📚 Recursos adicionales

📖 **Documentación Oficial**:

- [Git - Book](https://git-scm.com/book/en/v2)
- [Pro Git (Libro Gratuito)](https://git-scm.com/book/en/v2)
- [Atlassian Git Guide](https://www.atlassian.com/git/tutorials)

🎥 **Videos Recomendados**:

- [Git y GitHub desde cero - Fazt](https://youtu.be/DL-uxHE3tRk)
- [Mastering Git - Academind](https://youtu.be/Uszj_k0DGsg)

🔗 **Cursos Recomendados**:

- [Git y GitHub - Platzi](https://platzi.com/cursos/git-github/)
- [Learn Git - Codecademy](https://www.codecademy.com/learn/learn-git)

🎯 Conclusión

Git es una herramienta esencial para cualquier programador. Dominarlo no solo te ayudará en entrevistas, sino que **hará que tu flujo de trabajo sea más eficiente y profesional**. 🚀

Si esta guía te ayudó, ¡no dudes en ⭐ darle un **Star** al repositorio!

> 💡 "El conocimiento es poder, pero compartirlo es aún más poderoso." - Anónimo